Day 136: A DCGAN-Style Convolutional GAN, on MNIST and the Junction Dataset

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 4 | Complete study guide

Watch: DC-GAN Explained! (Connor Shorten) | Fractionally Strided or Transposed Convolution through an example (Aparajita Ojha)

Script: `day136_dcgan.py` · every number in this guide is copied by program from `day136_run_output.txt`, the real output of that script.

Day 134's GAN rebuilt from convolutions: the discriminator shrinks the image with strided convolutions, the generator grows one with transposed convolutions and batch normalisation, all written from scratch. On MNIST, with about the same number of generator weights, the DCGAN spread its output over the digits a little more evenly than the dense GAN (commonest digit 0.151 and 0.158 against 0.177 and 0.261) and matched the real digits' smoothness (average jump between neighbouring pixels 0.0625 against real 0.0650), at about 20 to 27 times the training time. Without batch normalisation, both seeds collapsed onto a single output for several epochs and one never recovered. On Week 7's junction images, where a generated image can be checked exactly, neither GAN learned the task in 100 epochs: at most 0.058 of the DCGAN's samples were valid junctions, against 0.990 of real images.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

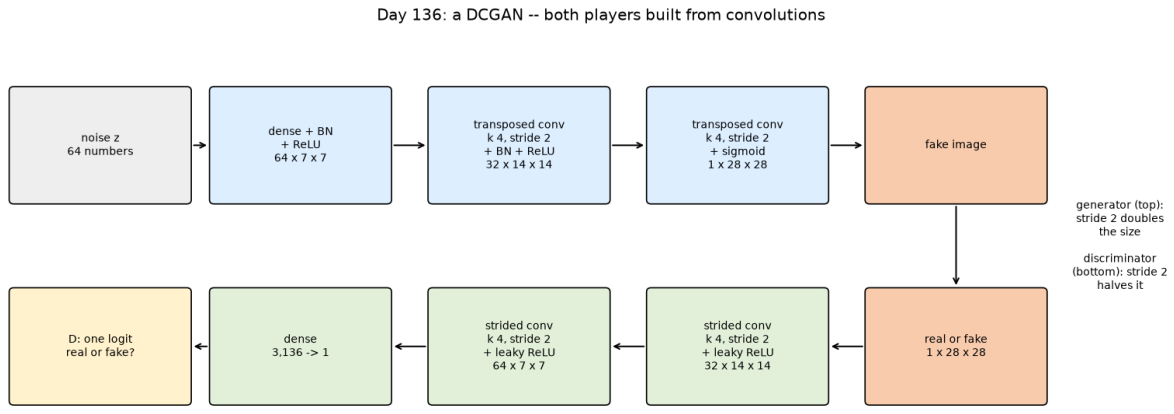
A **DCGAN** (deep convolutional GAN, Radford, Metz and Chintala, 2015) is a GAN whose two players are convolutional networks. A **strided convolution** moves its kernel s pixels at a time, so stride 2 halves the height and width; the discriminator uses two of them to shrink a 28 × 28 image to 7 × 7. A **transposed convolution** goes the other way: each input pixel paints a whole k × k patch of the output, patches s pixels apart, overlaps added; the generator uses two of them to grow 7 × 7 maps into a 28 × 28 image.

Because a convolution uses the same small kernel at every position, the network does not have to learn a stroke separately for every place in the image. Day 134's dense generator gave every pixel its own weights.

REAL-WORLD SCENARIO

A sign painter who owns a set of letter stencils works faster and more neatly than one who paints every letter freehand: the same stencil (kernel) is used wherever that shape is needed.

SYSTEM DIAGRAM



The two players. Top: the generator, from 64 noise numbers to a 28 × 28 image. Bottom: the discriminator.

Today's part for the week's build

`day136_exercise.py` is the convolutional machinery (im2col, col2im, Conv, ConvT, BN, the layer stack, both players and one round of the game), checked by hand: the shapes, the adjoint test, the overlap counts of a 4 × 4 kernel at stride 2, one gradient against finite differences, and 200 rounds on MNIST.

ACTUAL OUTPUT

```
strided convolution (2, 1, 28, 28) -> (2, 4, 14, 14); transposed convolution back to (2, 1, 28,
    28)
<conv(X), Y> = 4.6809728382; <X, convT(Y)> = 4.6809728382 (the same: one is the other's adjoint)
kernel 4, stride 2, all-ones weights on a 3 x 3 input of ones; each output pixel counts its
    contributions:
[[1 2 2 2 2 1]
 [2 4 4 4 4 2]
 [2 4 4 4 4 2]
 [2 4 4 4 4 2]
 [2 4 4 4 4 2]
 [1 2 2 2 2 1]]
d loss / d ConvT weight [1, 2, 3, 0]: backward 0.0032860936; finite difference 0.0032860936
rounds 1-50: D(real) 0.783, D(fake) 0.129
rounds 51-100: D(real) 0.962, D(fake) 0.025
rounds 101-150: D(real) 0.977, D(fake) 0.022
rounds 151-200: D(real) 0.976, D(fake) 0.031
500 generated images after 200 rounds: shape (500, 784), ink (average pixel) 0.127; real digits
    0.130
```

Data: MNIST (70,000 handwritten digits, cached by Day 126; labels used only by the judge) and Week 7's junction images, made by Day 47's generator at 28 × 28. Day 134's losses and AdamGAN, Day 116's softmax regression (the judge) and Day 119's distances are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: shapes, the adjoint, and overlaps

DEFINITION — start here, no prior knowledge assumed

Three facts about the new layers. The output sizes follow from the kernel k, the stride s and the padding p. The transposed convolution is the **adjoint** of the convolution with the same weights: for any X and Y, <conv(X), Y> = <X, convT(Y)>. That is why its forward pass is the convolution's backward pass. And with all weights set to 1, a transposed convolution shows how many input pixels paint each output pixel.

REAL-WORLD SCENARIO

Rolling a paint roller over a wall in strokes that overlap: if the overlap is uneven, some stripes get two coats and others one.

MATHEMATICS

strided convolution: $H_{out} = (H + 2p - k) / s + 1$ *transposed:* $H_{out} = (H - 1)\, s - 2p + k$

adjoint: $\langle conv(X), Y \rangle = \langle X, conv^{T}(Y) \rangle$ *for every X, Y*

IMPLEMENTATION

```
class ConvT:
    def __init__(self, cin, cout, k, s, p, rng):
        self.W = (rng.randn(cin, cout, k, k) * np.sqrt(2.0 / (cin * k * k / (s *
            s)))).astype(np.float32)
        self.b = np.zeros(cout, np.float32); self.k, self.s, self.p = k, s, p
    def params(self):
        return [(self.W, self.b)]
    def forward(self, X):
        m, C, H, W = X.shape; k, s, p = self.k, self.s, self.p
        Ho, Wo = (H - 1) * s + k, (W - 1) * s + k
        Xm = X.transpose(0, 2, 3, 1).reshape(-1, C); self.cache = (Xm, H, W)
        Y = col2im(Xm @ self.W.reshape(C, -1), (m, self.W.shape[1], Ho, Wo), k, s, H, W)
        return Y[:, :, p:Ho - p, p:Wo - p] + self.b[None, :, None, None]
```

ACTUAL OUTPUT

```
discriminator: (1, 28, 28) -> (32, 14, 14) -> (64, 7, 7)   strided convolution: (H + 2p - k) / s + 1 = (28 + 2 - 4) / 2 + 1
    = 14
generator:     (64, 7, 7) -> (32, 14, 14) -> (1, 28, 28)   transposed: (H - 1) s - 2p + k = 6 x 2 - 2 + 4 = 14
adjoint check, same weights: <conv(X), Y> = 15.8151776735;  <X, convT(Y)> = 15.8151776735  (relative gap 6.7e-16)
kernel 4, stride 2: output (14, 14); contributions per pixel, top-left 4 x 4:
       1  2  2  2
       2  4  4  4
       2  4  4  4
       2  4  4  4
   away from the edges: from 4 to 4 contributions
kernel 3, stride 2: output (13, 13); contributions per pixel, top-left 4 x 4:
       1  2  1  2
       2  4  2  4
       1  2  1  2
       2  4  2  4
   away from the edges: from 1 to 4 contributions
kernel 4 with stride 2 gives every inner pixel the same number of contributions; kernel 3 alternates 1, 2, 4:
an uneven grid built into the layer itself, the source of the 'checkerboard' pattern
Actual output from this lesson's run.
```

What it means

The two inner products agree to a relative gap of 6.7e-16: rounding error. With a 4 × 4 kernel at stride 2, every pixel away from the border gets exactly 4 contributions. With a 3 × 3 kernel at stride 2, neighbouring pixels get 1, 2 or 4: an uneven grid built into the layer, which the literature (Odena et al., 2016) names as a source of checkerboard patterns. Even with kernel 4, section 3's epoch-1 images show a faint grid; a likely reason, not tested here, is that the four positions within each 2 × 2 block use different kernel weights until training evens them out.

WHY CHOOSE THIS?

Kernel 4 with stride 2 is the DCGAN default because the overlap is even.

WHY NOT THIS? (TRADEOFFS)

Even overlap does not guarantee even weights; some designs upsample first and then convolve instead.

2. Correctness: the gradient through both players

DEFINITION — start here, no prior knowledge assumed

The generator's gradient has to pass back through the discriminator's two strided convolutions, then through two transposed convolutions and two batch normalisations. Twelve entries of each generator layer are checked against finite differences, in float64.

REAL-WORLD SCENARIO

Checking a long chain of gears by turning the last one a tiny amount and measuring how far the first one moves.

MATHEMATICS

$$\text{relative error} = |\text{numerical} - \text{backward}| \,/\, (|\text{numerical}| + |\text{backward}|), \quad \text{numerical} = (L(w + h) - L(w - h)) \,/\, 2h,\ h = 10^{-5}$$

ACTUAL OUTPUT

```
   layer  entries checked  worst relative error  largest |bias gradient|
   Dense                6               7.6e-09                  3.6e-18
      BN               12               2.6e-07                  1.1e-02
   ConvT                6               7.7e-09                  1.9e-17
      BN               12               3.6e-10                  3.6e-02
   ConvT               12               1.9e-10                  8.8e-02
worst over the generator: 2.6e-07 (the gradient passes back through D's two strided convolutions
    first)
the Dense and first ConvT biases have zero gradient: the batch normalisation right after them
    subtracts the
batch mean, so a constant added before it changes nothing (those entries are left out of the
    relative error)
Actual output from this lesson's run.
```

What it means

Worst relative error 2.6e-07. The first draft of this check reported 1.9e-05 for one layer, which looked like a bug. It was not: the largest bias gradient of the Dense layer and of the first ConvT is 3.6e-18 and 1.9e-17, zero up to rounding. Both layers are followed by batch normalisation, which subtracts the batch mean, so adding a constant before it changes nothing. A relative error between two numbers that are both rounding noise means nothing, so those entries are now reported separately.

WHY CHOOSE THIS?

A bias before batch normalisation is provably useless; real DCGANs drop it.

WHY NOT THIS? (TRADEOFFS)

Kept here so the layers stay identical whether batch normalisation is on or off (section 4).

3. MNIST: the dense GAN against the DCGAN

DEFINITION — start here, no prior knowledge assumed

Both GANs are trained exactly as on Day 134 (20,000 images, batches of 128, 20 epochs, AdamGAN with b1 0.5), two seeds each. Every epoch, 2,000 fixed noise vectors are judged by Day 134's judge: the rarest and the commonest digit's share (0.100 each if even), its confidence, the distance to the nearest training image, and a new measure of smoothness, the average jump between neighbouring pixels.

REAL-WORLD SCENARIO

Two print shops, one setting every dot by hand and one using stamps, given the same orders and judged by the same inspector.

SPECIFICATION

Dense	G [64, 256, 784], D [784, 256, 1], Day 134's players rebuilt as a layer stack
DCGAN	G 64 → 64×7×7 → 32×14×14 → 1×28×28; D two strided convolutions (32, 64 channels) → 1
Measured	every epoch on 2,000 fixed noise vectors

IMPLEMENTATION

```
def net_gan_step(G, D, oG, oD, X, rng, k):
    n = len(X); F = G.forward(rng.standard_normal((n, k)).astype(np.float32))
    Af = D.forward(F); Ar = D.forward(X)
    ld, gr, gf = d134.d_loss(Ar, Af)
    g1, _ = D.backward(gr); D.forward(F); g2, _ = D.backward(gf)
    oD.step(D.net, [(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)])
    F = G.forward(rng.standard_normal((n, k)).astype(np.float32))
    lf = D.forward(F); lg, gl = d134.g_loss(lf)
    _, dF = D.backward(gl); gG, _ = G.backward(dF)
    oG.step(G.net, gG)
```

ACTUAL OUTPUT

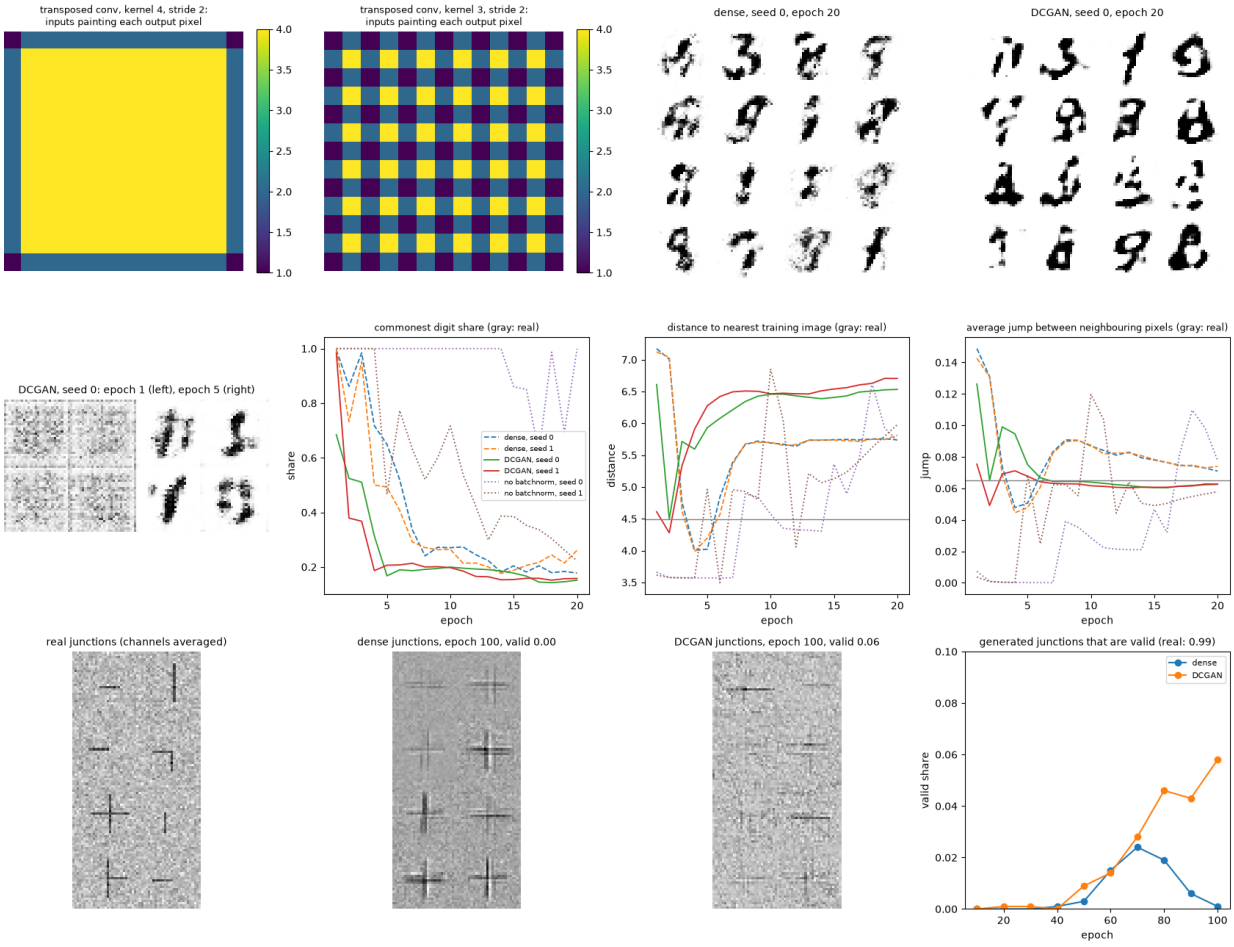
```
real test digits: rarest 0.077, commonest 0.136, judge's confidence 0.8757, nearest training image 4.49, pixel-to-pixel
    jump 0.0650
dense: generator 218,128 numbers, discriminator 201,217
DCGAN: generator 243,489 numbers (64 -> 64 x 7 x 7 -> 32 x 14 x 14 -> 1 x 28 x 28), discriminator 36,513
both: 20,000 training images, batches of 128, 20 epochs, AdamGAN (lr 0.0002, b1 0.5), the non-saturating loss;
each epoch, on 2,000 fixed noise vectors: the judge's rarest and commonest digit share (0.100 each if even),
its confidence, the distance to the nearest training image, and the average jump between neighbouring pixels
dense, seed 0:  epoch  D(real)  D(fake)  rarest  commonest  confidence  nearest    jump
                    1    0.860    0.064   0.000      0.996      0.8303     7.17  0.1488
                    2    0.781    0.133   0.000      0.862      0.5272     7.01  0.1311
                    3    0.840    0.226   0.000      0.985      0.6446     4.76  0.0739
                    5    0.934    0.210   0.000      0.649      0.5721     4.02  0.0507
                   10    0.828    0.160   0.043      0.271      0.6720     5.69  0.0869
                   15    0.825    0.172   0.026      0.204      0.7224     5.74  0.0780
                   20    0.798    0.199   0.035      0.177      0.7242     5.74  0.0710
                (78 s)
dense, seed 1:  epoch  D(real)  D(fake)  rarest  commonest  confidence  nearest    jump
                    1    0.855    0.060   0.000      0.999      0.9300     7.12  0.1426
                    2    0.788    0.123   0.000      0.732      0.5394     7.04  0.1308
                    3    0.849    0.234   0.000      0.953      0.6450     4.62  0.0686
                    5    0.935    0.214   0.000      0.492      0.5331     4.20  0.0478
                   10    0.831    0.157   0.031      0.265      0.6751     5.70  0.0874
                   15    0.825    0.172   0.027      0.188      0.7234     5.74  0.0783
                   20    0.800    0.191   0.053      0.261      0.7354     5.81  0.0739
                (52 s)
DCGAN, seed 0:  epoch  D(real)  D(fake)  rarest  commonest  confidence  nearest    jump
                    1    0.910    0.057   0.000      0.684      0.5702     6.61  0.1263
                    2    0.971    0.043   0.000      0.524      0.4947     4.49  0.0651
                    3    0.968    0.034   0.000      0.510      0.4891     5.71  0.0991
                    5    0.921    0.068   0.024      0.168      0.6836     5.93  0.0751
                   10    0.741    0.255   0.066      0.200      0.7465     6.46  0.0639
                   15    0.702    0.295   0.056      0.177      0.7527     6.41  0.0605
                   20    0.691    0.307   0.049      0.151      0.7500     6.53  0.0625
                (1573 s)
DCGAN, seed 1:  epoch  D(real)  D(fake)  rarest  commonest  confidence  nearest    jump
                    1    0.906    0.061   0.000      0.984      0.7249     4.61  0.0754
                    2    0.972    0.049   0.000      0.379      0.5142     4.28  0.0491
                    3    0.974    0.030   0.004      0.367      0.5931     5.33  0.0691
                    5    0.873    0.129   0.034      0.206      0.7214     6.28  0.0676
                   10    0.732    0.268   0.050      0.198      0.7595     6.46  0.0617
                   15    0.675    0.323   0.057      0.154      0.7577     6.54  0.0607
                   20    0.671    0.326   0.051      0.158      0.7618     6.70  0.0627
                (1401 s)
dense, seed 0, final spread of digits: 0.03 0.10 0.09 0.13 0.18 0.06 0.07 0.13 0.11 0.10
dense, seed 1, final spread of digits: 0.06 0.06 0.08 0.10 0.26 0.06 0.05 0.13 0.12 0.07
DCGAN, seed 0, final spread of digits: 0.05 0.07 0.09 0.15 0.13 0.11 0.06 0.11 0.09 0.15
DCGAN, seed 1, final spread of digits: 0.05 0.06 0.08 0.13 0.15 0.10 0.07 0.12 0.09 0.16
Actual output from this lesson's run.
```

What it means

First a check: the dense GAN, rebuilt as a layer stack, reproduces Day 134's printed numbers for b1 0.5 exactly (epoch 1: D(real) 0.860, confidence 0.8303, nearest 7.17).

The DCGAN has about the same number of generator weights (243,489 against 218,128) but a far smaller discriminator (36,513 against 201,217). By epoch 20 its digits are spread a little more evenly (commonest 0.151 and 0.158 against 0.177 and 0.261), it reached every digit earlier (rarest above zero by epoch 5: 0.024 and 0.034), the judge is a little more confident (0.7500 and 0.7618 against 0.7242 and 0.7354), and its pixel-to-pixel jump matches real digits (0.0625, 0.0627; real 0.0650) where the dense GAN's stays rougher (0.0710, 0.0739).

Two numbers do not favour it. Its images sit farther from the nearest training image (6.53 and 6.70, dense 5.74 and 5.81, real test digits 4.49): farther is less copying but also less like real digits, and these measures cannot tell which. And it cost 1573 s and 1401 s against 78 s and 52 s, measured while another long training run shared the CPU; an earlier run of the same script measured 1309 s and 1258 s against 79 s and 54 s, with every other number identical. Day 138 scores both with standard metrics.



Top: the overlap counts for kernels 4 and 3; the dense and DCGAN samples at epoch 20. Middle: the DCGAN at epochs 1 and 5; the commonest digit's share, the nearest-image distance and the pixel jump during training, with batch normalisation (sections 3) and without (section 4). Bottom: real junctions, both GANs' junctions, and the valid share during training (section 5).

WHY CHOOSE THIS?

Shared kernels: smoother images and a smaller discriminator.

WHY NOT THIS? (TRADEOFFS)

About 20 to 27 times slower here, in NumPy; the gains on these measures are small.

4. The DCGAN without batch normalisation

DEFINITION — start here, no prior knowledge assumed

The same two seeds with both batch normalisation layers removed from the generator. Nothing else changes.

REAL-WORLD SCENARIO

A choir without a conductor keeping everyone at the same volume: one loud singer can drown the rest.

MATHEMATICS

$$x' = (x - \mu_{batch}) / \sqrt{(\sigma^2_{batch} + 10^{-5})}, \quad y = \gamma x' + \beta \quad \textit{(per channel)}$$

IMPLEMENTATION

```
class BN:
    def __init__(self, c):
        self.W = np.ones(c, np.float32); self.b = np.zeros(c, np.float32)
    def params(self):
        return [(self.W, self.b)]
    def forward(self, X):
        ax = (0, 2, 3) if X.ndim == 4 else (0,); sh = (1, -1, 1, 1) if X.ndim == 4 else (1, -1)
        mu = X.mean(ax, keepdims=True); inv = 1 / np.sqrt(X.var(ax, keepdims=True) + 1e-5)
        Xh = (X - mu) * inv; self.cache = (Xh, inv, ax, sh)
        return Xh * self.W.reshape(sh) + self.b.reshape(sh)
```

ACTUAL OUTPUT

```
no batchnorm, seed 0:  epoch  D(real)  D(fake)  rarest  commonest  confidence  nearest    jump
                           1    0.911    0.122   0.000      1.000      0.6099     3.66  0.0073
                           2    0.993    0.028   0.000      1.000      0.6116     3.58  0.0010
                           3    0.998    0.005   0.000      1.000      0.6117     3.57  0.0004
                           5    1.000    0.001   0.000      1.000      0.6118     3.57  0.0002
                          10    1.000    0.000   0.000      1.000      0.6796     4.57  0.0285
                          15    1.000    0.000   0.000      0.860      0.5660     5.36  0.0467
                          20    0.994    0.005   0.000      1.000      0.5697     5.73  0.0777
                       (2668 s); final spread of digits: 0.00 0.00 0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00
no batchnorm, seed 1:  epoch  D(real)  D(fake)  rarest  commonest  confidence  nearest    jump
                           1    0.910    0.157   0.000      1.000      0.6116     3.61  0.0037
                           2    0.995    0.019   0.000      1.000      0.6117     3.58  0.0008
                           3    0.999    0.003   0.000      1.000      0.6118     3.57  0.0004
                           5    0.998    0.002   0.000      0.466      0.4455     4.96  0.0686
                          10    0.999    0.000   0.000      0.717      0.2895     6.85  0.1197
                          15    0.847    0.156   0.006      0.385      0.7702     5.13  0.0490
                          20    0.737    0.254   0.024      0.223      0.7853     5.98  0.0579
                       (2181 s); final spread of digits: 0.03 0.22 0.12 0.08 0.15 0.04 0.08 0.18 0.02 0.07
Actual output from this lesson's run.
```

What it means

Both seeds collapsed at once. For the first epochs the commonest digit's share is 1.000 and the pixel jump falls to 0.0002 and 0.0004: one smooth, featureless image for every input. Its distance to the nearest training image, 3.57, is even smaller than a real test digit's (4.49): a blurred average shape is close to many training images in pixel distance, so 'nearest' alone can reward collapse. D(fake) fell to 0.000. Seed 1 broke out after epoch 3 and ended with every digit present (rarest 0.024); seed 0 never did: at epoch 20 every image is judged a 5. Two seeds are not a rate, but with batch normalisation neither seed ever produced a featureless image (its lowest pixel jump was 0.0491).

WHY CHOOSE THIS?

Batch normalisation kept both runs out of the single-image collapse here.

WHY NOT THIS? (TRADEOFFS)

It makes each image depend on the rest of its batch; generation here uses batches of 500.

5. Junctions: can each GAN draw straight arms of the right length?

DEFINITION — start here, no prior knowledge assumed

Week 7's junction images (Days 43–47): a centre pixel with 1, 2, 3 or 4 straight arms of length 8 pointing up, down, left or right, in three channels, with noise added. Unlike a handwritten digit, a generated junction can be checked exactly. The checker builds all 375 perfect junctions (25 centre positions × 15 sets of arms), finds the closest one to an image, and measures the misfit on the 33 pixels of that centre's full cross. An image is **valid** if its misfit is below what 99% of real images reach.

REAL-WORLD SCENARIO

A ruler for a drawing test: a sketch of a face can only be judged by eye, but a drawing of a plus sign can be measured.

SPECIFICATION

Data	4,000 junctions, 1,000 each of 1–4 arms, 3 × 28 × 28, noise 0.4 (Week 7's generator, unchanged)
Players	Day 134's dense GAN and today's DCGAN, both with 3 channels and a linear output (the pixels are not in [0, 1])
Training	100 epochs, seed 0, AdamGAN (lr 0.0002, b1 0.5); 1,000 fixed samples checked every 10 epochs

IMPLEMENTATION

```
def read_junctions(F):
    I = F.reshape(len(F), 3, 784).mean(1)
    best = np.argmin((TEMPL ** 2).sum(1)[None, :] - 2 * I @ TEMPL.T, axis=1)
    misfit = (((I - TEMPL[best]) ** 2) * CROSS[best]).sum(1) / CROSS[best].sum(1)
    return misfit, ARMS[best]
def junction_stats(F, cut):
    err, arms = read_junctions(F); ok = err < cut
    h = np.bincount(arms[ok], minlength=5)[1:] / max(1, ok.sum())
    return float(ok.mean()), h, float(np.median(err))
```

ACTUAL OUTPUT

```
data: 4,000 junctions (1,000 each of 1, 2, 3 and 4 arms of length 8), 3 x 28 x 28, noise 0.4; pixel values from -2.20 to
    4.03
the arm counter on the real images: arms read correctly 1.0000; misfit median 0.164, 99th percentile 0.270; valid if misfit
    < 0.270
real images: valid share 0.990; split by arms 1-4: 0.242 0.252 0.255 0.252
dense (generator 621,104, discriminator 602,625 numbers), seed 0, linear output, 100 epochs:
      epoch  D(real)  D(fake)  valid  median misfit   arms 1-4 among the valid     jump
         10    0.942    0.008  0.000          0.839   0.000 0.000 0.000 0.000    1.431
         20    0.978    0.004  0.000          0.799   0.000 0.000 0.000 0.000    1.260
         30    0.975    0.007  0.000          0.678   0.000 0.000 0.000 0.000    1.062
         40    0.978    0.009  0.001          0.649   1.000 0.000 0.000 0.000    0.910
         50    0.964    0.010  0.003          0.588   0.333 0.667 0.000 0.000    0.755
         60    0.958    0.018  0.015          0.561   0.133 0.867 0.000 0.000    0.611
         70    0.964    0.023  0.024          0.551   0.042 0.958 0.000 0.000    0.492
         80    0.975    0.023  0.019          0.551   0.053 0.947 0.000 0.000    0.380
         90    0.982    0.026  0.006          0.590   0.500 0.500 0.000 0.000    0.275
        100    0.987    0.022  0.001          0.649   0.000 1.000 0.000 0.000    0.198
      (165 s)
```

ACTUAL OUTPUT

```
DCGAN (generator 244,515, discriminator 37,537 numbers), seed 0, linear output, 100 epochs:
      epoch  D(real)  D(fake)  valid  median misfit   arms 1-4 among the valid     jump
         10    0.791    0.122  0.000          0.680   0.000 0.000 0.000 0.000    0.749
         20    0.826    0.124  0.001          0.645   1.000 0.000 0.000 0.000    0.404
         30    0.842    0.095  0.001          0.694   1.000 0.000 0.000 0.000    0.402
         40    0.851    0.111  0.000          0.684   0.000 0.000 0.000 0.000    0.458
         50    0.833    0.132  0.009          0.596   1.000 0.000 0.000 0.000    0.484
         60    0.825    0.132  0.014          0.557   0.929 0.071 0.000 0.000    0.477
         70    0.815    0.149  0.028          0.536   0.929 0.071 0.000 0.000    0.479
         80    0.808    0.162  0.046          0.505   0.870 0.130 0.000 0.000    0.485
         90    0.792    0.150  0.043          0.529   0.953 0.047 0.000 0.000    0.494
        100    0.787    0.171  0.058          0.521   0.862 0.138 0.000 0.000    0.499
      (2145 s)
real images' jump: 0.526
Actual output from this lesson's run.
```

What it means

The checker works on real images: it reads the number of arms correctly in 1.0000 of them, and 0.990 of the first 2,000 pass the validity cut (0.270, set as the 99th percentile of real misfits).

Neither GAN learned to draw junctions in 100 epochs. The dense GAN's valid share peaked at 0.024 (epoch 70) and fell back to 0.001; the DCGAN's was still rising at the end, to 0.058. Every valid junction from either GAN had 1 or 2 arms; no 3- or 4-arm junction ever passed. Their median misfits ended at 0.649 and 0.521, against 0.164 for real images. The dense GAN's discriminator won throughout (D(real) 0.987, D(fake) 0.022 at epoch 100); the DCGAN's game stayed closer.

The pixel jump is dominated here by the added noise. The DCGAN's (0.499) is close to the real images' (0.526); the dense GAN's fell to 0.198, far smoother than real data, and its samples in the figure look like crosses with arms drawn several times side by side. A likely reason the task is hard, not tested here: an arm must be exactly 8 pixels long and exactly straight, while the discriminator only has to find one flaw.

WHY CHOOSE THIS?

An exact checker turns 'does it look right?' into a number, and showed that both GANs failed at a task a template solves.

WHY NOT THIS? (TRADEOFFS)

One seed and 100 epochs; the DCGAN was still improving, so a longer run might go further. That was not run.

Interview preparation — technical questions

Q: How does a strided convolution change the image size?

A: $H_{out} = (H + 2p - k) / s + 1$: with k 4, s 2, p 1, 28 × 28 becomes 14 × 14.

Q: What is a transposed convolution, exactly?

A: The convolution's input-gradient used as a layer: its adjoint. Here <conv(X), Y> and <X, convT(Y)> agreed to 6.7e-16.

Q: Why kernel 4 with stride 2?

A: Every inner output pixel gets the same number of contributions (4); kernel 3 gives 1, 2 or 4.

Q: Why did a gradient check report 1.9e-05 at first?

A: Biases before batch normalisation have zero true gradient; a relative error between two rounding-noise numbers is meaningless.

Q: What did batch normalisation do here?

A: Without it both seeds produced one featureless image for several epochs and one stayed collapsed; with it neither did.

Interview preparation — theoretical questions

Q: Why should convolutions help a generator?

A: A kernel is reused at every position, so a stroke is learned once; the DCGAN's images matched real smoothness (0.0625 against 0.0650).

Q: Is the DCGAN clearly better here?

A: Only slightly on these measures (commonest digit 0.151 and 0.158 against 0.177 and 0.261), at about 20 to 27 times the cost.

Q: Why can 'distance to the nearest training image' mislead?

A: A collapsed, blurred image was closer (3.57) than real test digits (4.49).

Q: How can a generated image be checked exactly?

A: Use data with a known structure: junctions matched against all 375 perfect templates; real 0.990 valid, the GANs at most 0.058.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Convolutions changed the generator's images in measurable ways: smoother, spread over the digits a little more evenly, judged with a little more confidence. Batch normalisation was what kept the convolutional GAN out of single-image collapse. Neither GAN learned to draw exact junctions in 100 epochs; an exact checker turned that failure into a number.

- Adjoint check 6.7e-16; worst gradient error 2.6e-07 (biases before batch normalisation have zero gradient).
- MNIST, epoch 20: commonest digit 0.151 and 0.158 (DCGAN) against 0.177 and 0.261 (dense); pixel jump 0.0625 against real 0.0650.
- Without batch normalisation: one featureless image for several epochs in both seeds; seed 0 ended with every image a 5.
- Junctions, 100 epochs: valid share at most 0.024 (dense) and 0.058 (DCGAN), against 0.990 for real images; never 3 or 4 arms.

Suggested watch order, if starting from zero

- 1. DC-GAN Explained! (Connor Shorten) — a walk through the DCGAN paper's architecture rules.
- 2. Fractionally Strided or Transposed Convolution through an example (Aparajita Ojha) — a worked example of a transposed (fractionally strided) convolution.

Day 137 preview

- **Conditional GANs**: the label goes to both players, so the generator can be asked for a 7.